파인튜닝 전체 여정 (v1 → v6)

"도메인별 영상 스크립트 자동화봇" 이 완성되기까지의 6 단계 기록

이 문서의 목적

하나의 챗봇 지시문이 실전 테스트를 거치며 어떻게 6 번의 라운드에 걸쳐 다듬어졌는지, 각 라운드에서 무엇이 문제였고 왜 그랬는지, 어떻게 고쳤는지를 순서대로 정리했습니다. 파인튜닝이 '한 번에 끝나지 않고 반복되는 과정'이라는 것을 보여주는 실제 사례입니다.

v1 (최초 버전)

발견된 문제: — (아직 실전 테스트 전)

원인: —

수정 내용: 역할·구조·스타일 규칙 4 가지로 뉴스·브랜드·트렌드 톤앤매너 변환 지시문 작성 (로봇저널리즘 자동화봇과 같은 방식)

결과: 실전 테스트 시작 — 스킨큐어 산다화 소재로 첫 실행

v2 (사실 혼합 방지)

발견된 문제: 뉴스 도메인 결과에 "자사 생명과학연구소에서 개발한 다중층 리포좀 공법"이라는 문장 등장

원인: 리포좀 공법(사실)과 생명과학연구소(사실)는 각각 진짜였지만, 원자료에서 후자는 협력사(바이오스펙트럼)의 것이며 전혀 다른 브랜드(노스테) 이야기였음 — 서로 다른 문단의 사실을 잘못 연결 (conflation)

수정 내용: 규칙 5 번 추가: "서로 다른 문단의 사실을 하나로 합쳐 원문에 없는 소유·인과 관계를 새로 만들어내지 않는다"

결과: 사실 혼합 오류 해결. 다만 재테스트 중 새로운 문제 2 개 발견(날짜, 이모지)

v3 (날짜·이모지 규칙 보강)

발견된 문제: ① 뉴스 도메인에 원자료에 없던 "14 일" 이라는 날짜 등장 ② 트렌드 도메인 이모지가 규칙(1~2 개)보다 훨씬 많은 8 개 사용됨

원인: ① 날짜를 지어내지 말라는 규칙 자체가 없었음 ② "1~2 개" 라는 표현이 모호해서 모델이 문장마다 이모지를 붙여버림

수정 내용: ① "소재에 날짜가 없으면 날짜 표현 자체를 생략할 것" 규칙 추가 ② "전체 글 통틀어 총 4 개 이내" 로 명확한 상한 설정

결과: 두 문제 모두 재테스트에서 해결 확인

v4 (영상 스크립트로 전면 개편)

발견된 문제: 지시문이 '글 분량' 만 다뤄서, 실제 영상 제작에 필요한 러닝타임·구성이 반영 안 됨. "2 분 내외로 작성해줘" 라고 직접 요청하자 Gemini 가 분량 계산을 위해 파이썬 코드를 실행하다 5 분 이상 멈춤

원인: 지시문에 러닝타임·시간 배분 규칙이 없어 모델이 즉흥적으로 코드 실행 경로를 선택함

수정 내용: 도메인별 표준 러닝타임 확정(뉴스 2 분·브랜드 1 분·트렌드 30 초) + 구간별 시간 배분표 + 타임코드 출력 규칙 추가. 규칙 6 번 신설: "러닝타임 계산을 위해 코드를 실행하지 않는다"

결과: 코드 실행 우회 문제 해결. 다만 타임코드 자체가 최종 결과에서 빠지는 새 문제 발견

v5 (타임코드 규칙에 예시 추가)

발견된 문제: "앵커멘트" 처럼 구간 이름만 나오고 [00:00-00:20] 같은 실제 타임코드가 계속 빠짐

원인: (추정) 규칙 설명만으로는 부족해서 구체적 예시가 필요하다고 판단

수정 내용: 규칙 4 번에 완성된 예시 문장 삽입: "앵커멘트 [00:00-00:20]\n 스킨큐어가 오늘..."

결과: 동일 문제 재발 — 프롬프트에서 "규칙을 엄격히 준수해달라" 고 강조해도 해결 안 됨. 지시문 설계 자체의 문제로 재확인됨

v6 (대괄호 → 소괄호로 근본 원인 해결)

발견된 문제: 같은 타임코드 누락 문제 지속

원인: 사용자가 결정적 단서 포착: "생성되는 과정에서는 타임코드가 보이다가, 완료되면서 없어진다." → 지시문 문제가 아니라 Gemini 화면이 대괄호 [ ]를 마크다운 링크·인용 표시로 오인해 렌더링 단계에서 지워버리는 것으로 진단

수정 내용: 모든 타임코드 표기를 대괄호에서 소괄호로 전환: "앵커멘트 (00:00-00:20)"

결과: 완전 해결 — 최종 테스트에서 6 개 규칙(사실혼합·날짜·이모지·코드실행방지·타임코드·러닝타임) 전부 통과

핵심 교훈 3 가지

① 파인튜닝은 반복 과정이다

6 번의 라운드 중 단 한 번도 ‘한 번에 완벽하게’ 해결되지 않았습니다. 매 라운드마다 기존 문제를 고치면 새 문제가 드러나거나, 고쳤다고 생각한 게 재발했습니다. 이건 실패가 아니라 파인튜닝이라는 과정의 정상적인 모습입니다.

② 문제의 ‘종류’에 따라 해결책이 다르다

- 사실 문제(v2, v3-날짜) → 지시문에 판단 규칙을 추가
- 형식 준수 문제(v3-이모지) → 모호한 표현을 명확한 숫자로 교체
- 모델의 즉흥적 판단 문제(v4) → 특정 행동을 명시적으로 금지
- 도구의 렌더링 특성 문제(v6) → 지시문이 아니라 출력 문법(기호) 자체를 바꿔 우회

v5 에서 ‘규칙에 예시를 추가’하는 것으로 해결하려 했던 시도가 실패한 이유는, 애초에 문제의 종류를 잘못 짚었기 때문입니다. 진짜 원인은 지시문이 아니라 도구의 화면 렌더링 방식에 있었습니다.

③ 결정적 진단은 결국 사람의 관찰에서 나왔다

v6 의 해결은 AI 가 스스로 찾아낸 게 아니라, 사용자가 “생성 중에는 보였다가 완료 후 사라진다”는 미세한 현상을 직접 관찰하고 알려주었기 때문에 가능했습니다. 이건 4 주차 원칙(AI 생성물 검수는 사람의 몫)이 파인튜닝 과정 자체에도 그대로 적용된다는 것을 보여줍니다 — AI 가 결과물을 만드는 동안에도, 무엇이 왜 이상한지 알아채는 것은 여전히 사람의 역할입니다.